

Day 12 - Forms in Depth and Debugging with DevTools

Day 12 — Forms in Depth and Debugging with DevTools

Objectives

- Use HTML5's built-in form validation attributes, understanding what the browser checks automatically for free
- Style validation states (`:valid/invalid`) to give visitors clear feedback
- Style form controls consistently, working around inconsistent default browser appearances
- Build a systematic process for debugging a layout that "just isn't working," using DevTools properly rather than guessing

HTML5 built-in validation — checks the browser performs automatically

Recall from Day 2 that you built a form without worrying yet about validating what visitors typed into it. HTML5 provides several attributes that make the *browser itself* enforce basic rules, with zero extra code required:

```
<input type="text" required>
<input type="email" required>
<input type="number" min="1" max="100">
<input type="text" pattern="[A-Za-z]{3,}" title="At least 3 letters, no numbers">
<input type="text" minlength="3" maxlength="20">
```

- `required` — the form cannot be submitted at all while this field is empty; the browser automatically shows its own built-in message and focuses the field if a visitor tries to submit anyway.
- `type="email"` — the browser checks that the entered text at least loosely resembles a valid email address (contains an @ and a domain-like portion) before allowing submission.
- `min/max` (on `type="number"`) — restricts the field to a numeric range.
- `pattern` — a regular expression (a precise pattern-matching syntax, outside this course's scope to teach in full) the entered text must match; `title` provides the tooltip text shown if it doesn't match.
- `minlength/maxlength` — restricts how many characters can be typed.

This built-in validation is a genuinely useful first line of defense, but it is never sufficient on its own for anything that matters — a visitor can always disable JavaScript, use a tool to submit a request directly without ever loading your page's HTML at all, or simply have an old browser that doesn't support some of these attributes. Real applications always *also* validate again on the server, after the form is actually submitted, treating any client-side validation (browser-enforced, as covered here) purely as a nicer, faster experience for well-behaved visitors — never as an actual security or data-integrity guarantee. This course doesn't cover building that server side, but the principle is worth internalizing now, before you're ever tempted to trust client-side validation alone.

Styling validation states with `:valid` and `:invalid`

These are pseudo-classes (recall Day 9's `:hover`/`:focus`) that match an input based on whether its *current* content currently satisfies its own validation rules:

```
input:invalid {
  border-color: red;
}
input:valid {
  border-color: green;
}
```

One subtlety worth knowing: an empty **required** field is considered `:invalid` by default, which means, without extra care, every required field on your form shows as an alarming red border the instant the page loads, before a visitor has even had a chance to type anything — usually not the experience you actually want. A common, better pattern only shows the invalid styling once a visitor has actually interacted with (and left) the field:

```
input:not(:placeholder-shown):invalid {
  border-color: red;
}
```

`:not(...)` is a pseudo-class that inverts whatever selector you put inside it — `:not(:placeholder-shown)` matches an input whose placeholder text is currently NOT showing, which happens precisely once a visitor has typed *something* into it (even something that turns out to be invalid) — a reasonably good, commonly-used approximation of "the visitor has actually interacted with this field," achieved with pure CSS and no extra code.

Styling form controls consistently

Recall from Day 11 that browsers apply inconsistent default styling — this shows up especially strongly on form elements: buttons, checkboxes, and select dropdowns can look meaningfully different by default across Chrome, Firefox, and Safari. A common baseline reset for text-like inputs:

```
input[type="text"],
input[type="email"],
input[type="password"],
textarea,
select {
  font-family: inherit; /* by default, some browsers use a different font for form controls than the page */
  font-size: inherit;
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
  width: 100%;
}
```

Notice the selector here: `input[type="text"]` is an **attribute selector** — it matches an `<input>` element specifically when its `type` attribute equals `"text"`, letting you target one specific kind of input without needing an extra class on every single one. `font-family: inherit;` and `font-size: inherit;` are worth calling out specifically: by default, many browsers render form controls using the *operating system's* own default UI font rather than whatever font-family you set on `<body>`, which is precisely why form elements can look visually "out of place" compared to the rest of a page unless you explicitly tell them to inherit your page's own

font choices instead.

Debugging a layout that "isn't working" — a systematic process, not guessing

Every beginner eventually hits a moment where a layout simply doesn't look the way they expected, and the instinct is often to randomly change values and reload, hoping something works. Here's a genuinely more effective, systematic process, using DevTools:

- 1 **Open DevTools and click the misbehaving element directly** (right-click → Inspect). This immediately shows you the *exact* HTML for that element, and, critically, its box model diagram.
- 2 **Check the Styles panel** for every rule currently affecting that element, in the order the browser is applying them, with strikethrough text showing rules that are being overridden by something more specific (recall Day 3's specificity) — this immediately tells you if some *other* rule you forgot about is winning unexpectedly.
- 3 **Check the "Computed" tab** (next to Styles) to see the final, actual resolved value of any property, after all overrides have been applied — genuinely useful when you're not sure which of several competing rules actually won.
- 4 **Check the box model diagram** (usually visible in the Styles or Computed panel) to see the actual rendered content/padding/border/margin sizes — this immediately reveals whether an unexpected size is due to **box-sizing** (Day 3), unexpected padding, or margin collapsing (Day 4).
- 5 **Temporarily disable rules** by unchecking them in the Styles panel (rather than editing your actual file) to isolate exactly which single rule is causing the unexpected behavior — turn things off one at a time until the problem disappears, and you've found your culprit.
- 6 **Only once you've identified the actual cause** using the steps above, go edit your real **.css** file — not before, since guessing-and-reloading without this diagnostic process wastes far more time than this systematic approach, especially as your stylesheets grow larger than what you've worked with so far in this course.

This process — inspect, read the Styles panel, check the box model, isolate by disabling rules, THEN edit — is, in practice, how real, experienced developers debug CSS every single day. It's worth deliberately practicing it in today's exercise rather than skipping straight to the solution file.

Exercises

Open **starter.html** and **starter.css**. This exercise has two parts: build the validation styling, AND deliberately debug a layout bug using the process above (see the specific instructions inside the files). Check your work against **CHECKLIST.md**.